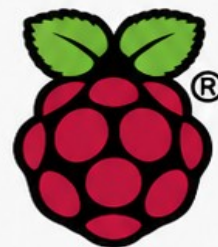


PROYECTO SERVIDOR DOCKER EN RASPBERRY PI



Despliegue de múltiples servicios en contenedores Docker sobre Raspberry Pi



Alumno:

Pablo Rodríguez Escobar



Curso:

2º SMR



Ciclo Formativo:

Sistemas Microinformáticos y Redes



Año:

2025 – 2026



Tutor:



PROYECTO DE FIN DE CICLO

C.F.G.M. Sistemas Microinformáticos y Redes

“ Un servidor eficiente, modular y escalable al alcance de todos. ”

SERVIDOR DOCKER EN RASPBERRY PI

Autor: Pablo Rodríguez Escobar

Curso: 2º SMR

Tutor: Juan limon

Fecha: _____

HOJA RESUMEN DEL PROYECTO

Título del proyecto

Servidor Docker en Raspberry Pi

Autor

Pablo Rodríguez Escobar

Tutor

Palabras clave

Castellano

Raspberry Pi, Docker, Docker Compose, servidor, contenedores, Linux, Nextcloud, WordPress, Plex.

Inglés

Raspberry Pi, Docker, Docker Compose, server, containers, Linux, Nextcloud, WordPress, Plex.

Resumen del proyecto

El proyecto consiste en convertir una Raspberry Pi en un servidor ligero utilizando Docker y Docker Compose. El objetivo principal es demostrar que es posible desplegar múltiples servicios de red sobre hardware económico y de bajo consumo energético.

Durante el desarrollo del proyecto se implementarán diferentes servicios en contenedores independientes, como WordPress, Nextcloud, Duplicati, Plex y Transmission. Además, se estudiará la viabilidad económica y técnica del sistema, así como la planificación, seguridad y mantenimiento del entorno.

El proyecto también incluye una fase previa de virtualización y pruebas mediante QEMU en Windows 10, permitiendo validar configuraciones antes de implementarlas en la Raspberry Pi real.

Indice

Resumen del proyecto.....	2
1. INTRODUCCIÓN.....	7
1.1 Introducción a la memoria.....	7
1.2 Descripción.....	7
1.3 Objetivos generales.....	7
1.4 Beneficios.....	8
1.5 Motivaciones personales.....	8
1.6 Estructura de la memoria.....	8
2. ESTUDIO DE VIABILIDAD.....	8
2.1 Introducción.....	8
2.1.1 Tipología y palabras clave.....	9
Tipología.....	9
Palabras clave.....	9
2.1.2 Descripción.....	9
2.1.3 Objetivos del proyecto.....	9
2.1.4 Clasificación de objetivos.....	10
2.1.5 Definiciones y acrónimos.....	10
2.1.6 Partes interesadas.....	10
2.2 Estudio de la situación actual.....	11
2.3 Requisitos del sistema.....	11
Requisitos hardware.....	11
Requisitos software.....	11
2.4 Alternativas estudiadas.....	11

Servidores dedicados.....	11
Equipos reciclados.....	11
Microservidores comerciales.....	12
Raspberry Pi.....	12
2.5 Planificación del proyecto.....	12
2.6 Evaluación de riesgos.....	12
Riesgos técnicos.....	12
Riesgos hardware.....	12
Riesgos de seguridad.....	12
2.7 Presupuesto.....	13
3. METODOLOGÍA DE DESARROLLO DEL PROYECTO.....	13
3.1 Metodología incremental.....	13
3.2 Reducción de riesgos.....	13
3.3 Validación temprana.....	13
4. ESTUDIO DE ALTERNATIVAS.....	14
4.1 Servidores tradicionales.....	14
Ventajas.....	14
Desventajas.....	14
4.2 Equipos reciclados.....	14
Ventajas.....	14
Desventajas.....	14
4.3 Raspberry Pi.....	14
Ventajas.....	14
Desventajas.....	14
5. OBJETIVOS PARCIALES.....	15
5.1 Instalación de Docker.....	15

5.2 Instalación de Docker Compose.....	15
5.3 Implementación de servicios.....	16
6. PLANIFICACIÓN TEMPORAL DEL PROYECTO.....	16
Fase 1: Análisis inicial.....	16
Fase 2: Emulación con QEMU.....	16
Instalación de QEMU.....	16
Fase 3: Validación del entorno.....	16
Fase 4: Implementación real.....	16
Fase 5: Documentación final.....	17
7. CONTROL DE VERSIONES.....	17
7.1 GitHub.....	17
7.2 Beneficios.....	18
8. MECANISMOS DE SEGURIDAD Y RECUPERACIÓN.....	18
8.1 Seguridad del sistema operativo.....	18
8.2 Seguridad en Docker.....	18
8.3 Seguridad de red.....	18
8.4 Copias de seguridad.....	18
9. ANÁLISIS DEL SISTEMA.....	19
9.1 Arquitectura general.....	19
10. DISEÑO DEL SISTEMA.....	19
10.1 Capa física.....	19
10.2 Capa del sistema operativo.....	19
10.3 Capa Docker.....	19
10.4 Docker Compose.....	20
11. IMPLEMENTACIÓN.....	20
11.1 Actualización del sistema.....	20

11.2 Instalación Docker.....	20
11.3 Configuración permisos.....	20
11.4 Verificación Docker.....	20
12. CONFIGURACIÓN DE DOCKER.....	20
12.1 Creación de servidor web NGINX.....	20
12.2 Creación HTML personalizado.....	21
13. IMPLEMENTACIÓN DE SERVICIOS.....	21
13.1 WordPress.....	21
13.2 Nextcloud.....	21
13.3 Duplicati.....	22
13.4 Plex.....	23
13.5 Transmission.....	23
14. CONFIGURACIÓN DE REDES Y ALMACENAMIENTO.....	24
14.1 Redes Docker.....	24
14.2 Volúmenes persistentes.....	24
14.3 Puertos utilizados.....	24
15. PRUEBAS REALIZADAS.....	24
15.1 Verificación Docker.....	24
15.2 Verificación de contenedores.....	25
16. RESULTADOS OBTENIDOS.....	25
17. CONCLUSIÓN.....	25
18. Sigüientes mejoras.....	26

1. INTRODUCCIÓN

1.1 Introducción a la memoria

En esta memoria se presenta el desarrollo de un proyecto cuyo objetivo es convertir una Raspberry Pi en un servidor ligero utilizando Docker. Gracias a su bajo coste, tamaño reducido y consumo energético mínimo, la Raspberry Pi se presenta como una excelente alternativa frente a servidores tradicionales.

El proyecto permitirá desplegar diferentes servicios dentro de contenedores independientes, facilitando así la administración del sistema y evitando conflictos entre aplicaciones.

El documento recoge todas las fases del proyecto, desde el análisis inicial y estudio de viabilidad hasta la implementación final del servidor.

1.2 Descripción

El proyecto consiste en instalar Raspberry Pi OS Lite sobre una Raspberry Pi y configurar Docker como plataforma de contenedorización.

Mediante Docker Compose se gestionarán diferentes servicios desplegados en contenedores independientes, permitiendo una arquitectura modular, sencilla de mantener y fácilmente ampliable.

Los servicios instalados son:

- WordPress
- Nextcloud
- Plex
- Duplicati
- Transmission

1.3 Objetivos generales

Los objetivos principales del proyecto son convertir una Raspberry Pi en un servidor funcional utilizando Docker como plataforma de contenedorización. También se pretende configurar Docker Compose para gestionar múltiples servicios de forma organizada y sencilla.

Otro objetivo importante es crear una arquitectura modular y escalable, permitiendo añadir nuevos servicios fácilmente en el futuro.

Además, se busca garantizar estabilidad, seguridad y bajo consumo energético, aprovechando las ventajas de la Raspberry Pi frente a servidores tradicionales.

1.4 Beneficios

Dentro de los beneficios principales que tiene es que es muy facil de instalar y utilizar, ya que sin tener muchos conocimiento, puede hacer bastantes cosas en la Raspberry Pi, otra de las cosas es que tiene un costo económico muy bajo.

1.5 Motivaciones personales

La motivación principal del proyecto es demostrar que es posible crear un servidor completo utilizando hardware económico y tecnologías modernas.

Además, este proyecto permite desarrollar conocimientos muy importantes dentro del sector informático, especialmente relacionados con virtualización, contenedorización y administración de sistemas.

1.6 Estructura de la memoria

La memoria se divide en varios apartados:

1. Introducción y descripción.
2. Estudio de viabilidad.
3. Diseño del sistema.
4. Implementación.
5. Configuración de servicios.
6. Seguridad.
7. Pruebas y resultados.
8. Conclusiones.

2. ESTUDIO DE VIABILIDAD

2.1 Introducción

En el estudio de viabilidad hemos podido determinar si el proyecto puede llevarse a cabo correctamente utilizando los recursos disponibles.

Se analizarán aspectos:

- Técnicos
- Económicos
- Funcionales

- Operativos

2.1.1 Tipología y palabras clave

Tipología

Proyecto técnico de administración de sistemas y despliegue de servicios mediante contenedores Docker.

Palabras clave

- Raspberry Pi
- Docker
- Linux
- Docker Compose
- Servidor
- Contenedores
- Nextcloud
- WordPress

2.1.2 Descripción

El sistema estará compuesto por una Raspberry Pi con Raspberry Pi OS Lite y Docker instalado.

Los diferentes servicios funcionarán mediante contenedores independientes.

Docker Compose permitirá gestionar:

- Redes
- Puertos
- Dependencias
- Volúmenes persistentes

2.1.3 Objetivos del proyecto

- Implementar Docker correctamente.
- Crear un servidor estable.
- Configurar servicios web.
- Implementar almacenamiento persistente.
- Permitir acceso remoto.
- Facilitar futuras ampliaciones.

2.1.4 Clasificación de objetivos

Objetivos técnicos

- Configurar Docker.
- Gestionar contenedores.
- Automatizar despliegues.

Objetivos económicos

- Reducir costes.
- Disminuir consumo energético.

Objetivos formativos

- Aprender Linux.
- Aprender Docker.
- Mejorar competencias digitales.

2.1.5 Definiciones y acrónimos

Raspberry Pi

Miniordenador de bajo coste.

Docker

Plataforma de contenedorización.

Docker Compose

Herramienta para gestionar múltiples contenedores.

OS

Sistema operativo.

ARM

Arquitectura de procesador utilizada por Raspberry Pi.

2.1.6 Partes interesadas

Las partes que pueden estar interesadas principalmente pueden ser:

- Centros educativos
- Usuarios domésticos
- Empresas pequeñas
- Administradores de sistemas

2.2 Estudio de la situación actual

Ahora mismo los servidores tradicionales tienen:

- Alto coste económico.
- Mayor consumo eléctrico.
- Mayor mantenimiento.
- Hardware más complejo.

Por lo que la Raspberry Pi representa una alternativa económica y eficiente.

2.3 Requisitos del sistema

Requisitos hardware

- Raspberry Pi 4/5
- Fuente de alimentación
- Tarjeta microSD o SSD
- Cable Ethernet

Requisitos software

- Raspberry Pi OS Lite
- Docker
- Docker Compose

2.4 Alternativas estudiadas

Servidores dedicados

Mayor potencia pero mucho más costosos.

Equipos reciclados

Mayor consumo eléctrico.

Microservidores comerciales

Más caros que Raspberry Pi.

Raspberry Pi

La opción elegida por:

- Precio reducido
- Bajo consumo
- Buen rendimiento

2.5 Planificación del proyecto

El proyecto se divide en varias fases:

1. Análisis inicial.
2. Virtualización con QEMU.
3. Instalación de Docker.
4. Configuración de servicios.
5. Pruebas.
6. Documentación final.

2.6 Evaluación de riesgos

Riesgos técnicos

- Fallos de configuración.
- Problemas de compatibilidad ARM.
- Errores en Docker.

Riesgos hardware

- Sobrecalentamiento.
- Fallos eléctricos.

Riesgos de seguridad

- Accesos no autorizados.
- Puertos expuestos.

2.7 Presupuesto

Concepto	Coste
Raspberry Pi	90 €
Fuente alimentación	15 €
Tarjeta microSD	20 €
Cable Ethernet	5 €
Caja y ventilación	25 €
TOTAL	155 €

3. METODOLOGÍA DE DESARROLLO DEL PROYECTO

3.1 Metodología incremental

Se ha utilizado una metodología incremental.

Esto permite:

- Detectar errores rápidamente.
- Validar configuraciones.
- Añadir servicios progresivamente.

3.2 Reducción de riesgos

La metodología incremental permite:

- Evitar errores graves.
- Mejorar estabilidad.
- Optimizar recursos.

3.3 Validación temprana

Cada servicio se prueba individualmente antes de integrar el sistema completo.

4. ESTUDIO DE ALTERNATIVAS

4.1 Servidores tradicionales

Ventajas

- Mucha potencia.
- Mayor capacidad.

Desventajas

- Coste elevado.
- Alto consumo.

4.2 Equipos reciclados

Ventajas

- Reutilización de hardware.

Desventajas

- Consumo elevado.
- Equipos ruidosos.

4.3 Raspberry Pi

Ventajas

- Bajo coste.
- Tamaño reducido.
- Muy bajo consumo.
- Fácil mantenimiento.

Desventajas

- Menor potencia.
- Limitaciones ARM.

5. OBJETIVOS PARCIALES

5.1 Instalación de Docker

Se instala Docker mediante:

`curl -fsSL https://get.docker.com | sh`

```
usuario@raspberrypi: ~
# curl -fsSL https://get.docker.com | sh
# Executing docker install script, commit: cee7f6a8ef

+ sh -c apt-get update -qq >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get install -y -qq \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
Selección del paquete ca-certificates previamente no seleccionado.
(leyendo la base de datos ... 133757 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../ca-certificates_20240203_all.deb ...
Desempaquetando ca-certificates (20240203) ...
Configurando ca-certificates (20240203) ...
Actualizando certificados en /etc/ssl/certs...
Añadiendo certificado: /etc/ssl/certs/ca-certificates.crt
rehash: warning: skipping ca-certificates.crt, it does not contain exactly one certificate or CRL
2 added, 0 removed; done.
Ejecutando hooks en /etc/ca-certificates/update.d...
done.
+ sh -c mkdir -p /etc/apt/keyrings
+ sh -c curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.gpg
+ sh -c chmod a+r /etc/apt/keyrings/docker.gpg
+ sh -c echo \deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian \
    $( /etc/os-release && echo "VERSION_CODENAME" ) stable > /etc/apt/sources.list.d/docker.list
+ sh -c apt-get update -qq >/dev/null
+ sh -c apt-get install -y -qq docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
+ sh -c docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from Docker Hub. (arm64v8)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
usuario@raspberrypi:~$
```

5.2 Instalación de Docker Compose

Docker Compose permite desplegar múltiples servicios simultáneamente.

```
usuario@raspberrypi1: ~/server
Archivo Editar Pestañas Ayuda
usuario@raspberrypi1:~/server$ cat docker-compose.yml
version: '3.8'

services:
  nextcloud:
    image: nextcloud
    ports:
      - "8081:80"
    volumes:
      - ./nextcloud:/var/www/html
    restart: always
  duplicati:
    image: lscr.io/linuxserver/duplicati
    ports:
      - "8200:8200"
    volumes:
      - ./duplicati:/config
      - ./backups:/backups
      - ./source:/source
    restart: always
```

5.3 Implementación de servicios

Se implementan:

- WordPress
- Nextcloud
- Plex
- Duplicati
- Transmission

6. PLANIFICACIÓN TEMPORAL DEL PROYECTO

Fase 1: Análisis inicial

Duración aproximada: 2 semanas.

Fase 2: Emulación con QEMU

Duración aproximada: 2 meses.

Instalación de QEMU

Comandos utilizados:

winget install SoftwareFreedomConservancy.QEMU

```
C:\Windows\system32>winget search qemu
```

Name	Id	Version	Match	Source
QEMU	SoftwareFreedomConservancy.QEMU	10.2.0	ProductCode: qemu	winget
WindowsSpyBlocker	CrazyMax.WindowsSpyBlocker	4.39.0.0	Tag: qemu	winget
Qemu Simple Boot	WsSolInfor.Qemu	1.3	Tag: qemu	winget
qemu-img	cloudbase.qemu-img	2.3.0	Tag: qemu	winget
QEMU guest agent	SoftwareFreedomConservancy.QEMUGuestAgent	110.0.2		winget
Qemu Boot Tester	WsSolInfor.QemuBootTester	24.2.25		winget
Virtio-win-guest-tools	RedHat.VirtIO	0.1.285-1	Tag: qemupcserial.inf	winget
Looking Glass	gnif.LookingGlass	B7	Tag: qemu-kvm	winget

Fase 3: Validación del entorno

Duración aproximada: 1 mes.

Fase 4: Implementación real

Duración aproximada: 3 semanas.

Fase 5: Documentación final

Duración aproximada: 2 semanas.

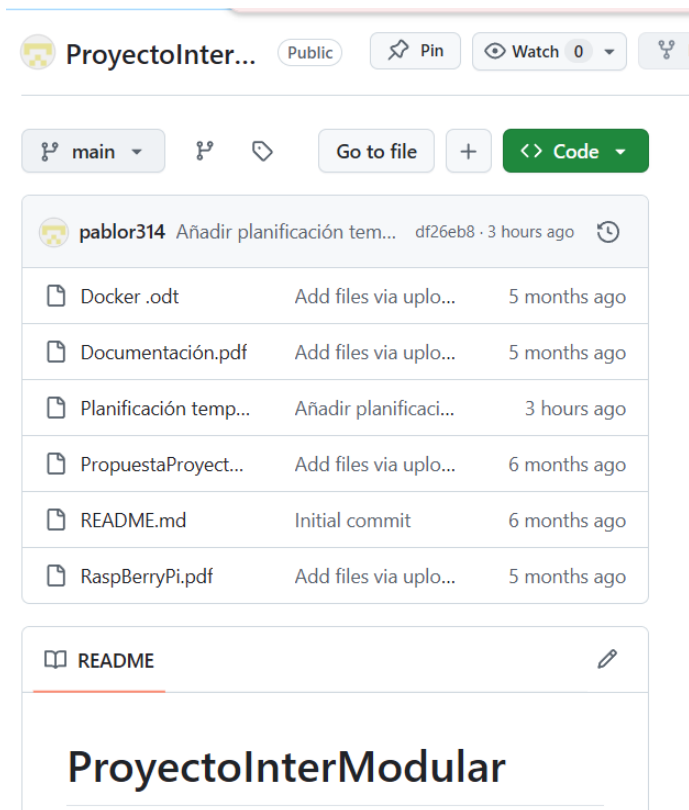
7. CONTROL DE VERSIONES

7.1 GitHub

Se utiliza GitHub como sistema de control de versiones.

Permite:

- Guardar cambios.
- Recuperar configuraciones.
- Compartir documentación.



7.2 Beneficios

- Seguridad.
- Historial de cambios.
- Acceso remoto.

8. MECANISMOS DE SEGURIDAD Y RECUPERACIÓN

8.1 Seguridad del sistema operativo

- Actualización periódica.
- Cambio de contraseñas.
- Usuarios limitados.

8.2 Seguridad en Docker

- Uso de imágenes oficiales.
- Contenedores aislados.
- Gestión de puertos.

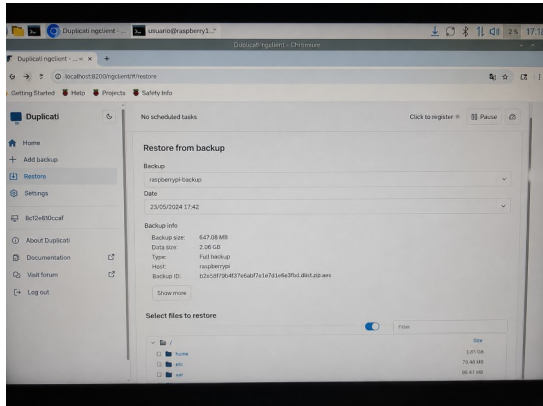
8.3 Seguridad de red

- Puertos mínimos.
- Red local.
- Firewall.

8.4 Copias de seguridad

Se realizan copias de:

- Configuración Docker.
- Datos persistentes.
- Documentación.



9. ANÁLISIS DEL SISTEMA

9.1 Arquitectura general

El sistema se divide en varias capas:

- Hardware
- Sistema operativo
- Docker
- Servicios

10. DISEÑO DEL SISTEMA

10.1 Capa física

La Raspberry Pi actúa como servidor principal

10.2 Capa del sistema operativo

Se utiliza Raspberry Pi OS Lite.

10.3 Capa Docker

Docker ejecuta todos los servicios en contenedores aislados.

10.4 Docker Compose

Docker Compose centraliza toda la configuración.

Ejemplo:

```
services:  
  wordpress:  
    image: wordpress
```

11. IMPLEMENTACIÓN

11.1 Actualización del sistema

```
sudo apt update  
sudo apt upgrade -y
```

11.2 Instalación Docker

```
curl -fsSL https://get.docker.com | sh
```

11.3 Configuración permisos

```
sudo usermod -aG docker $USER
```

11.4 Verificación Docker

```
docker run hello-world
```

12. CONFIGURACIÓN DE DOCKER

12.1 Creación de servidor web NGINX

```
docker run -d -p 80:80 --name web nginx
```

12.2 Creación HTML personalizado

`nano ~/web/index.html`

Contenido:

```
<h1>Todo funciona correctamente</h1>
<p>Conseguido!</p>
```

13. IMPLEMENTACIÓN DE SERVICIOS

13.1 WordPress

Configuración mediante Docker Compose.

Configuración MariaDB

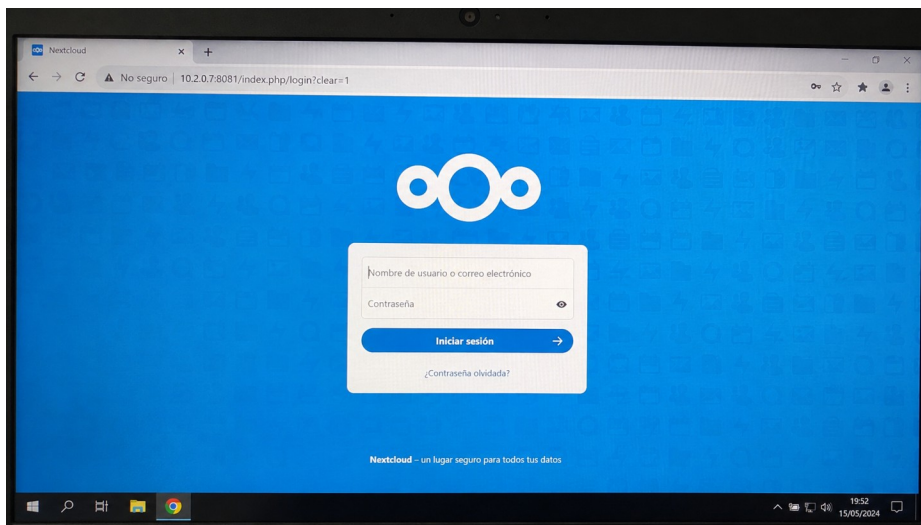
`image: mariadb`

Configuración WordPress

`image: wordpress`

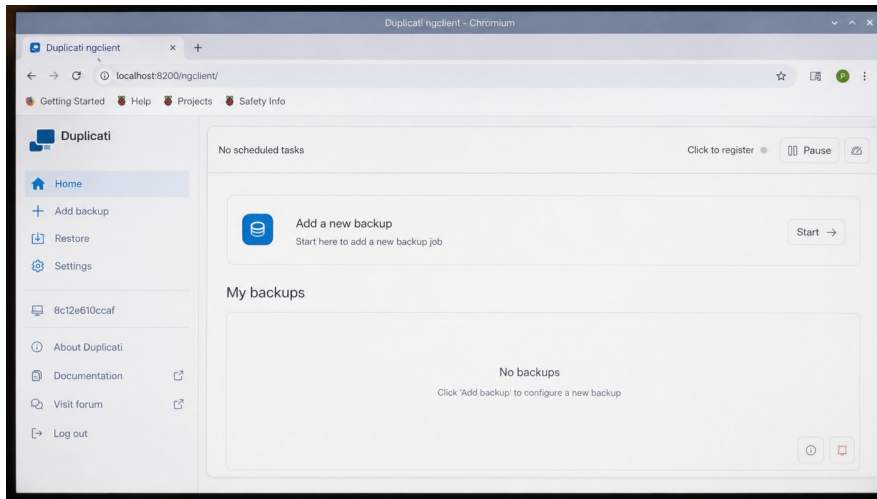
13.2 Nextcloud

Nextcloud es una plataforma de almacenamiento en la nube privada que permite guardar, sincronizar y compartir archivos entre distintos dispositivos, funciona de manera similar a Google Drive



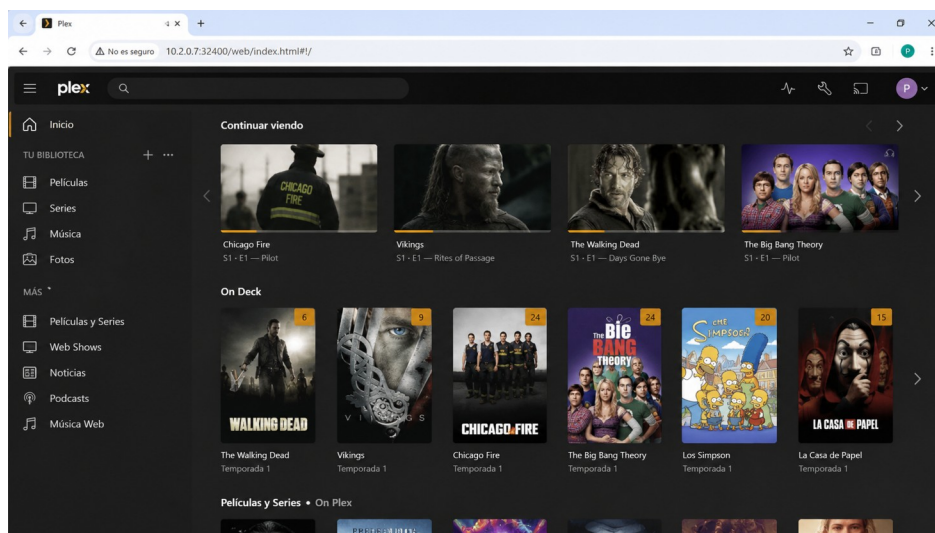
13.3 Duplicati

Duplicati es una herramienta de copias de seguridad automáticas, que nos permite realizar copias de seguridad cifradas de archivos y carpetas hacia discos externos, servidores o almacenamiento en la nube.



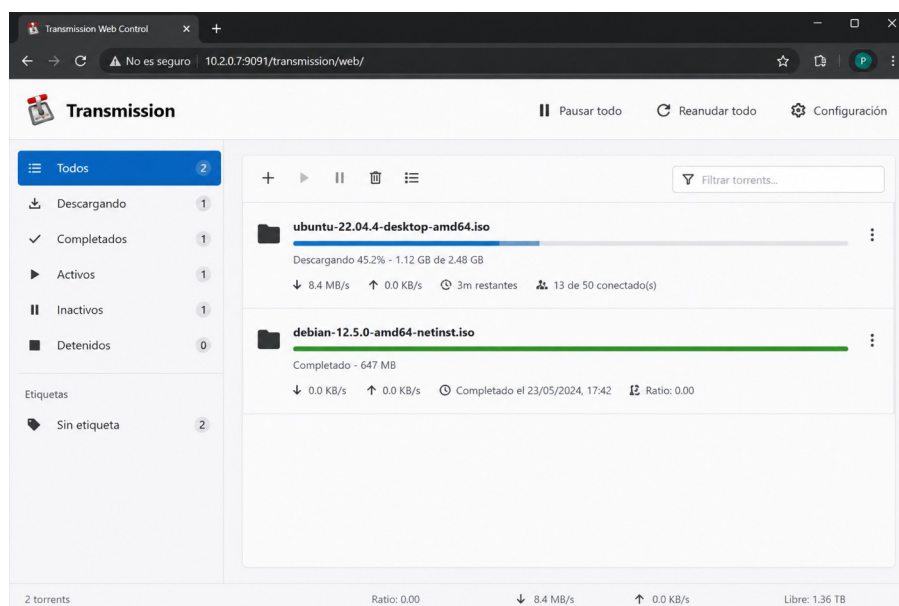
13.4 Plex

Plex es un servidor multimedia que organiza y reproduce películas, series, música y fotos desde una biblioteca central.



13.5 Transmission

Transmission es un cliente BitTorrent ligero utilizado para descargar archivos mediante la red P2P.



14. CONFIGURACIÓN DE REDES Y ALMACENAMIENTO

14.1 Redes Docker

Docker permite crear redes internas virtuales para facilitar la comunicación entre contenedores de forma segura y organizada. Gracias a estas redes, los servicios pueden conectarse entre sí utilizando nombres de contenedor en lugar de direcciones IP manuales.

14.2 Volúmenes persistentes

Los volúmenes persistentes permiten conservar la información almacenada por los contenedores incluso después de reinicios, actualizaciones o eliminación de los mismos.

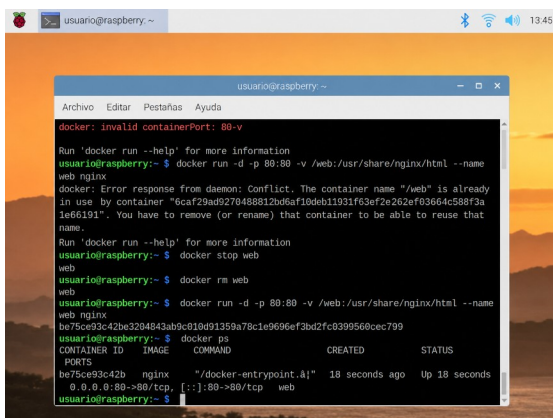
14.3 Puertos utilizados

Servicio	Puerto
WordPress	8080
Nextcloud	8081
Duplicati	8200
Transmission	9091
Plex	32400

15. PRUEBAS REALIZADAS

15.1 Verificación Docker

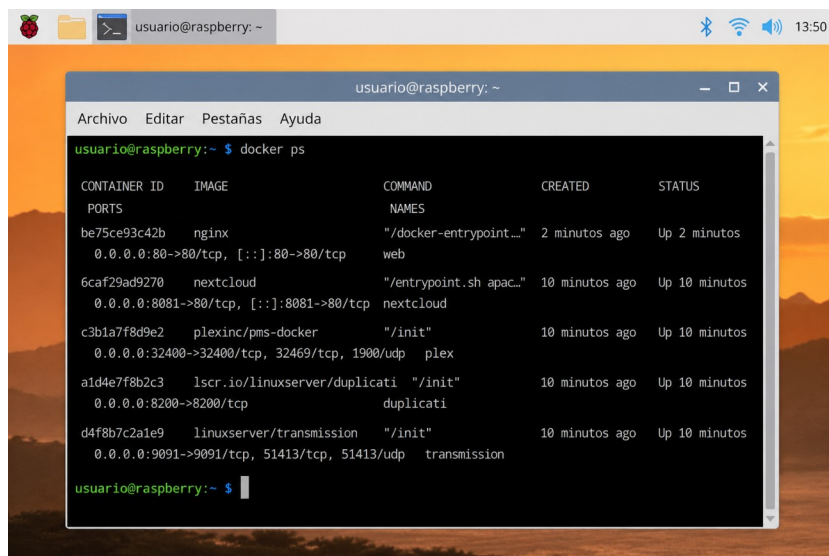
Se comprobó el correcto funcionamiento de Docker mediante el comando `docker ps`



```
usuario@raspberrypi ~  
docker: invalid containerPort: 80-v  
Run 'docker run --help' for more information  
usuario@raspberrypi:~$ docker run -d -p 80:80 -v /web:/usr/share/nginx/html --name web nginx  
docker: Error response from daemon: Conflict. The container name "/web" is already in use by container "6caf29ad9270488812bd6af0deb11931f63ef2e262ef03664c588f3a1e66191". You have to remove (or rename) that container to be able to reuse that name.  
Run 'docker run --help' for more information  
usuario@raspberrypi:~$ docker stop web  
web  
usuario@raspberrypi:~$ docker rm web  
web  
usuario@raspberrypi:~$ docker run -d -p 80:80 -v /web:/usr/share/nginx/html --name web nginx  
be75ce93c42be3204843ab9c018d91359a78c1e9696ef3bd2fc0399560cec799  
usuario@raspberrypi:~$ docker ps  
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS  
be75ce93c42b   nginx    "/docker-entrypoint.aj"  18 seconds ago  Up 18 seconds  
0.0.0.0:80->80/tcp, [::]:80->80/tcp  web  
usuario@raspberrypi:~$
```


15.2 Verificación de contenedores

Todos los contenedores iniciaron correctamente.



```
usuario@raspberrypi: ~  
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
be75ce93c42b	nginx	"/docker-entrypoint..."	2 minutos ago	Up 2 minutos
0.0.0.0:80->80/tcp, [::]:80->80/tcp		web		
6caf29ad9270	nextcloud	"/entrypoint.sh apac..."	10 minutos ago	Up 10 minutos
0.0.0.0:8081->80/tcp, [::]:8081->80/tcp		nextcloud		
c3b1a7f8d9e2	plexinc/pms-docker	"/init"	10 minutos ago	Up 10 minutos
0.0.0.0:32400->32400/tcp, 32469/tcp, 1900/udp		plex		
a1d4e7f8b2c3	lscr.io/linuxserver/duplicati	"/init"	10 minutos ago	Up 10 minutos
0.0.0.0:8200->8200/tcp		duplicati		
d4f8b7c2a1e9	linuxserver/transmission	"/init"	10 minutos ago	Up 10 minutos
0.0.0.0:9091->9091/tcp, 51413/tcp, 51413/udp		transmission		

```
usuario@raspberrypi: ~  
$
```

16. RESULTADOS OBTENIDOS

Los resultados que he obtenidos han sido bastante buenos

Se consiguió:

- Crear un servidor funcional.
- Implementar múltiples servicios.
- Mantener bajo consumo energético.
- Garantizar estabilidad.

17. CONCLUSIÓN

La Raspberry Pi, me a parecido un sistema de lo mas curioso, ya que yo no sabia todo lo que se podía hacer en una, y me a parecido muy completa, por otro lado, tambiénal ir buscando soluciones a los problemas presentados, ye aprendido a resolverlos y tambien a mejorar un poco mejor los comandos

18. Siguietes mejoras

- Añadir más servicios.
- Poner una VPN.
- Añadir monitorización avanzada.
- Mejorar seguridad.
- Poner IP publica, para poder